24CYS401 • SECURE SOFTWARE ENGINEERING

CAPSTONE PROJECT REPORT

SmartWaste Secure

IoT-Based Smart Waste Management and Secure Route Optimization Platform

STUDENT	REGISTER NUMBER	INSTITUTION	SEMESTER
Yeshwanth Raj S.	CH.SC.U4CYS23052	Amrita Vishwa Vidyapeetham - Chennai Campus	VII • 2023 Batch

COURSE	VERSION	DATE	CLASSIFICATION
24CYS401 Secure Software Engineering	1.0	08 October 2026	Academic Engineering Report

Core Engineering Principle

Security is applied across the smart-bin lifecycle: device identity is established, telemetry is authenticated and validated, suspicious inputs are detected, collection routes are protected by authorization, operational resources are controlled, sensitive actions are audited, and deployment is hardened at runtime.

DOCUMENT MAP

Table of Contents

Phase 1 - Agile Process and Development Approach
Phase 2 - Requirements Engineering / SRS
Phase 3 - Requirements Analysis and UML
Phase 4 - Data and Information Flow Modeling
Phase 5 - Software Architecture and Design Engineering
Phase 6 - User Interface Design
Phase 7 - Threat Modeling and Security Analysis
Phase 8 - Attack Tree and Security Architecture Refinement
Phase 9 - Product Backlog and Jira/Scrum
Phase 10 - Sprint Execution and Scrum Metrics
Phase 11 - Secure Development and Build Environment
Phase 12 - Secure Coding and Refactoring
Phase 13 - Docker and Kubernetes
Phase 14 - CI/CD and Security Testing
Phase 15 - Logging, Monitoring, Hardening and Secure Deployment
Phase 16 - Final Security Review
Appendices - Security Register, Test Matrix and References

FRONT MATTER

Abstract and Executive Summary

SmartWaste Secure is a secure-software engineering model for an IoT-based waste management platform. Smart bins periodically report fill level, location, temperature and sensor status. The platform validates this data, stores it, identifies suspicious sensor behaviour, generates optimized collection routes and allows authorized administrators to assign vehicles and drivers.

The engineering focus is the security boundary between untrusted IoT telemetry and trusted operational decisions. Device identity, secure transport, server-side validation, location consistency, route authorization, rate limiting, least privilege, auditing and hardened container/Kubernetes deployment are treated as design properties rather than optional add-ons.

> The integrated workflow used in this report is: Problem Statement -> Agile Process -> Requirements / SRS -> Use Cases -> ER / Data Model -> DFD / Trust Boundaries -> Architecture -> UI -> Threat Modeling -> Information Flow -> Vulnerability Analysis -> Attack Tree -> Backlog / Jira -> Sprint Execution -> Secure Build -> Secure Coding -> Docker / Kubernetes -> CI/CD / Testing -> Logging / Hardening -> Final Traceability.

Scope statement: this is an academic laboratory model. It is not presented as a certified municipal waste-management platform, and measured Jira metrics or deployment results are not claimed until captured from actual execution.

FRONT MATTER

List of Figures

FIGURE	TITLE
1	Proposed Smart Bin hardware and secure telemetry path
2	SmartWaste Secure UML Use Case Diagram
3	Scenario analysis - Generate and Assign a Route
4	SmartWaste Secure logical ER model
5	DFD Level 0 - Context Diagram
6	DFD Level 1 - Major processes and stores
7	DFD Level 2 - Telemetry Processing
8	DFD Level 2 - Route Management
9	Trust boundaries and security zones
10	Layered secure architecture
11	UI wireframes
12	Attack Tree
13	Kubernetes deployment architecture
14	Secure CI/CD pipeline

FRONT MATTER

List of Tables

TABLE	TITLE
Table 1	Mark Alignment
Table 2	Stakeholders and User Classes
Table 3	Functional Requirements
Table 4	Non-Functional Requirements
Table 5	Security Requirements
Table 6	Objectives Register
Table 7	Architecture Components and Controls
Table 8	Hardware and Sensor Specification
Table 9	Asset and CIA Classification
Table 10	STRIDE Threat Analysis
Table 11	Vulnerability Analysis
Table 12	Product Backlog

TABLE	TITLE
Table 13	Two-Sprint Plan and Metrics
Table 14	Secure Build Controls
Table 15	Security Test Matrix
Table 16	Final Requirement Traceability Matrix

FRONT MATTER

Mark Alignment

PHASE	ACTIVITY	MARKS
1	Agile Process and Development Approach	6
2	Requirements Engineering	7
3	Requirements Analysis and UML	7
4	Data and Information Flow Modeling	7
5	Software Architecture and Design Engineering	7
6	User Interface Design	5
7	Threat Modeling and Security Analysis	10
8	Attack Tree and Security Architecture Refinement	6
9	Product Backlog and Jira/Scrum	7
10	Sprint Execution and Scrum Metrics	7
11	Secure Development and Build Environment	6
12	Secure Coding and Refactoring	4
13	Containerized Development: Docker and Kubernetes	7
14	CI/CD and Security Testing	7
15	Logging, Monitoring, Hardening and Secure Deployment	5
16	Final Security Review	1
TOTAL	All phases	100

This phase-to-mark mapping follows the uploaded examination instructions.

PROJECT CONTEXT

Introduction and Problem Context

Background

Smart waste management combines IoT sensing, operational analytics, identity and access control, and continuous service availability. The engineering challenge is not simply to collect sensor values; the platform must ensure that untrusted device inputs cannot silently alter trusted operational decisions such as collection priority, bin location or route assignment.

Problem statement

The proposed system is an IoT-based waste management platform in which smart bins periodically send fill level, location, temperature and sensor status. The platform generates optimized collection routes, and administrators assign vehicles and drivers. The principal security challenges are sensor spoofing, location manipulation, unauthorized route modification, availability, device authentication and API security.

Project vision

SmartWaste Secure treats every IoT message and every high-impact route operation as security-sensitive. Device identity is established before telemetry is trusted; telemetry is validated before it can influence route optimization; route changes are authorized and audited; and the application is hardened through secure build, container and Kubernetes controls.

Engineering lifecycle

> **Integrated lifecycle:** Problem Statement -> Agile Process -> Requirements / SRS -> Use Cases -> ER / Data Model -> DFD / Trust Boundaries -> Architecture -> UI -> Threat Modeling -> Information Flow -> Vulnerability Analysis -> Attack Tree -> Product Backlog -> Jira / Scrum -> Sprint Execution -> Secure Build -> Secure Coding -> Docker / Kubernetes -> CI/CD / Testing -> Logging / Monitoring / Hardening -> Final Security Review.

Technology and toolchain

AREA	PLANNED TOOLING	PURPOSE
Requirements / report	HTML / PDF	SRS, traceability and final engineering report.
UML / DFD / ERD / Attack Tree	draw.io	Editable academic diagrams and security models.
IoT controller	ESP32-WROOM	Sensor acquisition, Wi-Fi connectivity and telemetry packaging.
Messaging	MQTT / Mosquitto	Laboratory device telemetry transport and topic access control.
Backend	Python Flask	REST API and security-sensitive business services.
Database	PostgreSQL	Persistent operational and audit data.
Testing	pytest / Hypothesis	Unit, integration, system and fuzz testing.
SAST / SCA / secrets	Bandit / pip-audit / Gitleaks	Static security, dependency and secret checks.
Container security	Docker / Trivy	Reproducible build and image security scanning.
Orchestration	Kubernetes / Minikube	Container deployment, runtime restrictions and resource controls.
Agile tracking	Jira Scrum	Epics, user stories, tasks, sprints and metrics.

PROJECT FOUNDATION

Objectives, Scope and Engineering Contributions

ID	OBJECTIVE
OBJ-01	Authenticate smart bins before accepting telemetry.
OBJ-02	Validate fill level, temperature, location and sensor-status data before operational use.
OBJ-03	Detect suspicious sensor or location behaviour.
OBJ-04	Generate optimized collection routes using validated bin data.
OBJ-05	Restrict route changes to authorized administrators.
OBJ-06	Provide auditable security and administrative activity.
OBJ-07	Demonstrate secure build, container, Kubernetes and CI/CD controls.
OBJ-08	Maintain end-to-end requirement-to-verification traceability.

Scope

In scope are smart-bin telemetry, device identity, API security, bin monitoring, route optimization, vehicle/driver assignment, audit logging, security alerts, UI, secure coding, Docker, Kubernetes, CI/CD and security testing. Physical manufacture of smart bins and production municipal infrastructure are outside the academic scope.

Engineering contributions

- Security model in which authenticated devices remain untrusted until telemetry passes validation and consistency checks.
- Route-management design that combines RBAC, object-level authorization, versioning and auditability.
- Cross-phase traceability linking requirements, DFD/trust boundaries, threats, vulnerabilities, backlog items, tests and deployment controls.

PHASE 1 • 6 MARKS

Phase 1 - Agile Process and Development Approach

Selected approach: Scrum with XP-oriented secure engineering practices. Two academic sprints are used, with continuous integration, code review, test-first thinking for critical modules and explicit security work in the backlog.

Agile Manifesto mapping

PRINCIPLE	SMARTWASTE APPLICATION
Individuals and interactions	Developer, system operator, security reviewer and faculty feedback inform security and usability decisions.
Working software	Each sprint delivers an executable increment: first secure telemetry foundations, then secure route operations and assurance.
Customer collaboration	Operational stakeholders validate bin monitoring, route generation and driver assignment workflows.

PRINCIPLE	SMARTWASTE APPLICATION
Responding to change	Threat analysis and test findings can alter backlog priority without breaking the overall security baseline.
Working software over documentation	Security documentation remains mandatory, but demonstrations and testable controls provide primary evidence.

Refactoring opportunities

OPPORTUNITY	BEFORE	AFTER / SECURITY BENEFIT
RO-01 - Route update logic	One controller handles authentication, validation, database update and logging.	Controller -> authorization service -> validation -> route service -> repository -> audit service. Separation makes authorization harder to bypass and simplifies testing.
RO-02 - Role checks	Scattered <code>if role == ...</ code> checks in controllers.	Central permission policy such as <code>route:update</code>. Fewer authorization gaps and clearer review.

Agile limitations and mitigations

RISK / LIMITATION	MITIGATION
Fast iterations can under-prioritize security.	Security acceptance criteria are attached to critical stories and protected by release gates.
Frequent changes can create inconsistent artifacts.	Requirement IDs, diagram vocabulary and the final traceability matrix are maintained as the common baseline.

PHASE 2 • 7 MARKS

Phase 2 - Requirements Engineering and SRS

The requirements baseline below is intended to be the authoritative requirements set for the later use cases, data models, threat analysis, backlog and verification activities. This is the same traceability principle emphasized by the reference report, where later models are explicitly derived from earlier requirement identifiers.

2.1 Stakeholders and user classes

ACTOR	PRIMARY INTERACTION	SECURITY CONCERN
Administrator	Manage smart bins, vehicles, drivers, routes and security settings.	Authentication, authorization, audit
Driver	View assigned route and update collection status.	Least privilege, assignment scoping
System Operator	Monitor platform and device health.	Operational access without admin privilege
Security Auditor	Review security events and audit evidence.	Read-only access to security records
Smart Bin	Periodically send fill, location, temperature and status telemetry.	Device identity, secure telemetry channel
Faculty / Evaluator	Review engineering evidence and consistency.	Traceability and verifiability

2.2 Functional requirements

ID	REQUIREMENT	PRIORITY
FR-01	Administrator shall authenticate before protected operations.	Critical
FR-02	Administrator shall register and manage smart bins.	High
FR-03	Smart bins shall authenticate before telemetry submission.	Critical
FR-04	System shall receive fill level, location, temperature and sensor status.	Critical
FR-05	System shall validate telemetry schema, type and acceptable ranges.	Critical
FR-06	System shall store telemetry with timestamp and device association.	High
FR-07	System shall display bin status and operational alerts.	High
FR-08	System shall detect suspicious or anomalous telemetry.	Critical
FR-09	System shall generate optimized collection routes.	Critical
FR-10	Administrator shall assign vehicles and drivers to routes.	High
FR-11	Driver shall view assigned route and collection sequence.	High
FR-12	Administrator shall modify or cancel routes subject to authorization.	Critical
FR-13	System shall maintain security-relevant audit records.	High
FR-14	System shall generate alerts for important anomalies and unauthorized actions.	High

2.3 Non-functional requirements

ID	AREA	SPECIFICATION	ACCEPTANCE DIRECTION
NFR-01	Security	External API communication shall use TLS.	No plaintext protected traffic.
NFR-02	Security	Protected operations shall enforce authentication and RBAC.	Server-side enforcement.
NFR-03	Availability	Core API should support health checks and controlled restart behavior.	Graceful recovery.
NFR-04	Performance	Normal telemetry ingestion should not block dashboard operations.	Responsive UI.
NFR-05	Scalability	Application should be container-ready and horizontally deployable.	Docker/Kubernetes compatible.
NFR-06	Maintainability	Business logic should be separated from API and data access.	Service/repository structure.
NFR-07	Auditability	Sensitive operations shall be traceable.	Actor, resource, time and outcome.
NFR-08	Usability	Dashboard shall present status, alerts and errors clearly.	Consistent navigation/feedback.
NFR-09	Reliability	Invalid telemetry shall not corrupt persistent state.	Validate before persistence.
NFR-10	Portability	Laboratory setup shall be reproducible.	Docker + Minikube.

2.4 Security requirements

ID	AREA	REQUIREMENT
SEC-01	Device Authentication	Every registered smart bin shall have a unique authenticated device identity.

ID	AREA	REQUIREMENT
SEC-02	Secure Transport	Telemetry and application traffic shall use TLS-protected channels.
SEC-03	Authorization	Route modifications and administrative functions require server-side RBAC/object authorization.
SEC-04	Input Validation	Telemetry and route inputs shall be schema, type, range and state validated.
SEC-05	Location Integrity	Coordinates shall be checked for valid ranges, impossible movement and suspicious change.
SEC-06	Replay Resistance	Telemetry shall use timestamp/sequence information to reject stale or repeated messages where implemented.
SEC-07	API Protection	Sensitive endpoints shall be rate-limited and return safe errors.
SEC-08	Secret Management	Credentials and keys shall not be hard-coded or committed to source control.
SEC-09	Least Privilege	The IoT device and application database account shall have only required permissions.
SEC-10	Audit	Login failures, privilege changes, route modifications and anomalous telemetry shall be logged.
SEC-11	Container Security	Application containers shall run without unnecessary privileges.
SEC-12	Kubernetes Security	Deployment shall apply non-root context, resource limits and restricted exposure.

2.5 CIA, authentication, authorization and audit objectives

Confidentiality: protect user/device credentials, route details, driver information, audit data and internal configuration from unauthorized disclosure.

Integrity: ensure telemetry, location, route and audit state cannot be silently altered through ordinary application paths.

Availability: keep telemetry and route-management services responsive using rate limiting, health checks and controlled resource usage.

Identity and accountability: authenticate devices and users, enforce RBAC/object-level authorization and record sensitive actions.

PHASE 3 • 7 MARKS

Phase 3 - Requirements Analysis and UML

3.1 UML Use Case Diagram

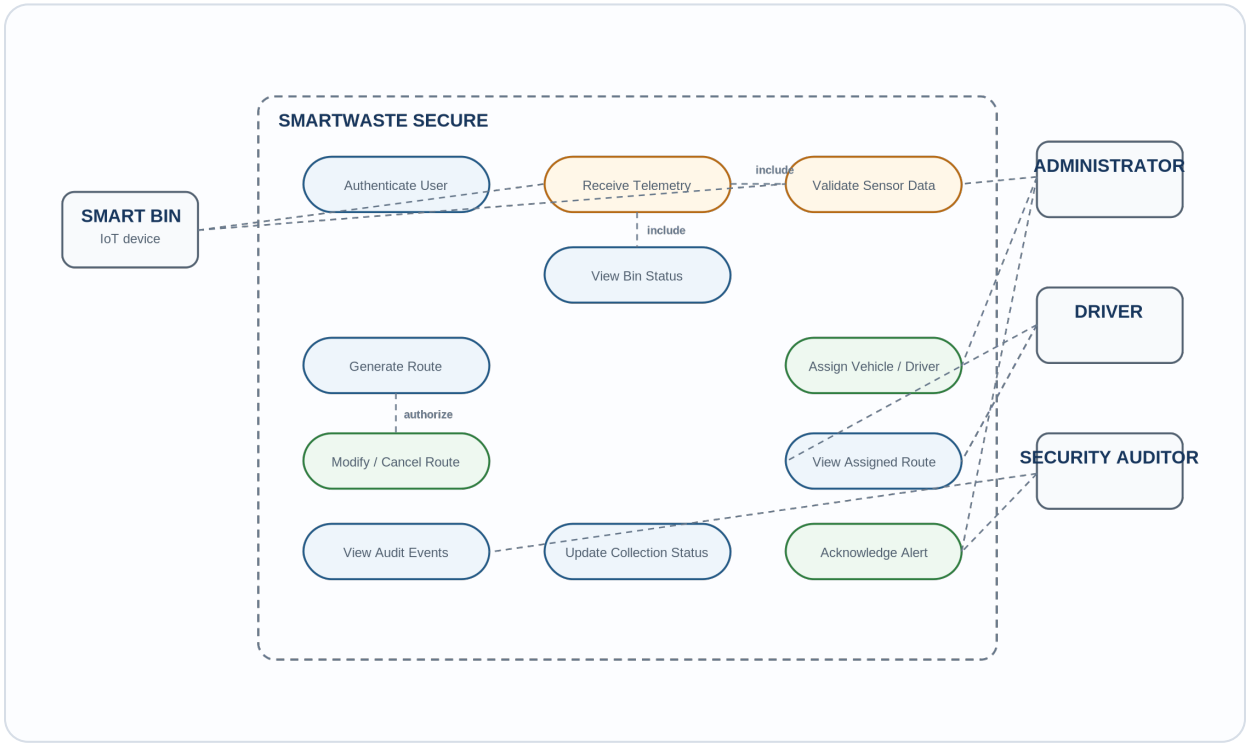


Figure 2 - SmartWaste Secure UML Use Case Diagram. The use cases separate device telemetry, administration, driver operations and security oversight.

3.2 Critical use case UC-01 - Submit Telemetry

FIELD	SPECIFICATION
Primary actor	Smart Bin
Preconditions	Device is registered and active; secure channel available.
Trigger	Periodic telemetry interval is reached.
Main flow	Authenticate device -> validate payload -> validate ranges -> check timestamp/sequence -> check location consistency -> accept -> persist -> update bin status -> audit/alert if required.
Alternative flow	If data is suspicious but not clearly invalid, store a security event and mark the telemetry for review without using it blindly for route optimization.
Exception flow	Unknown/revoked device -> reject; malformed data -> reject; invalid range -> reject; stale/replayed message -> reject where implemented.
Postconditions	Only accepted telemetry influences trusted application state; rejected or suspicious data is auditable.

3.3 Critical use case UC-02 - Modify Collection Route

FIELD	SPECIFICATION
Primary actor	Administrator
Preconditions	Administrator authenticated; route management permission present.
Trigger	Administrator requests a route update.

FIELD	SPECIFICATION
Main flow	Authenticate -> authorize permission -> validate route ID/state -> validate new values -> update version -> persist -> create audit event -> return safe response.
Alternative flow	Administrator saves a draft modification before final assignment.
Exception flow	Unauthorized role -> 403; invalid route -> 404; invalid state transition -> 400/409; validation failure -> 400.
Postconditions	Authorized change is persisted with version and audit history; unauthorized change has no state effect.

3.4 Scenario-based analysis - Generate and Assign a Route

1. Administrator signs in.
2. System authenticates and loads permitted operations.
3. Administrator requests route generation.
4. Eligible bins are retrieved.
5. Telemetry and location are validated.
6. Route engine generates an optimized sequence.
7. Administrator reviews the route.
8. Vehicle and driver are selected.
9. Assignment is validated and persisted.
10. Audit event is created.
11. Driver receives only the assigned route.

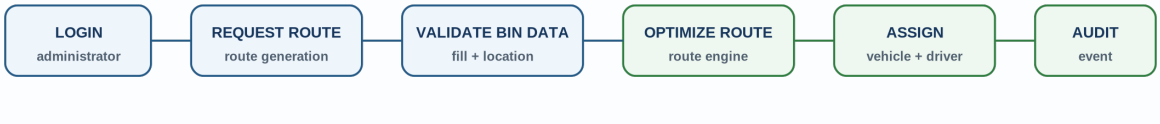


Figure 3 - Scenario-based analysis: Generate and Assign a Route.

PHASE 4 • 7 MARKS

Phase 4 - Data and Information Flow Modeling

4.1 ER model

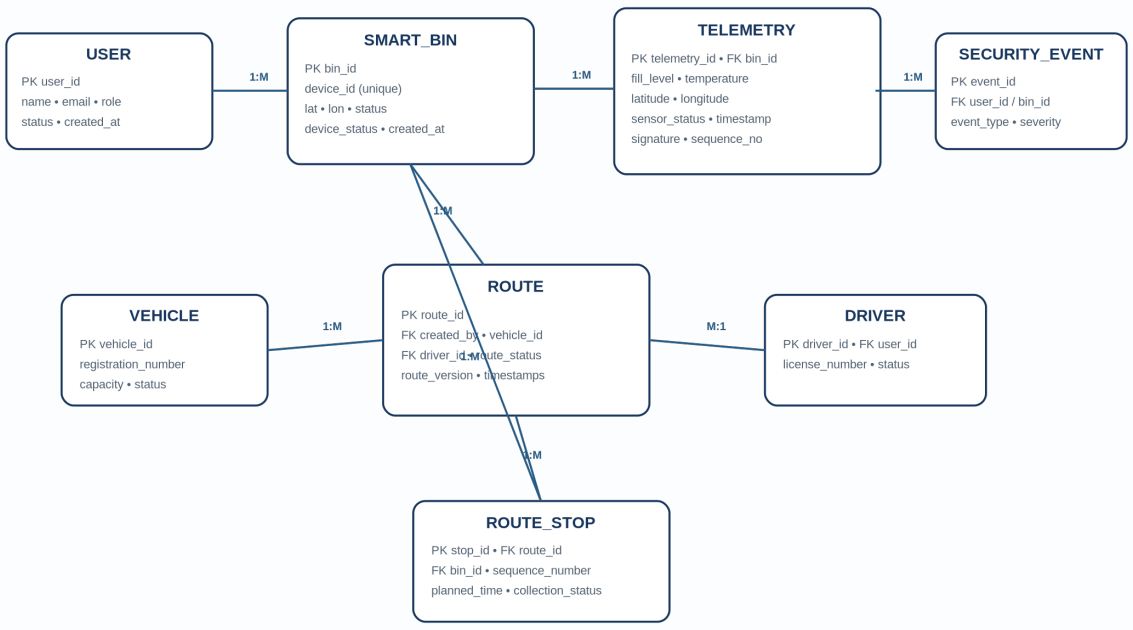


Figure 4 - SmartWaste Secure logical ER model. The same entities and relationships form the data baseline for the database, DFD and API models.

4.2 Core entities

ENTITY	PURPOSE	KEY FIELDS
USER	Identity, role and account state.	user_id, email, password_hash, role, status
SMART_BIN	Registered IoT device and nominal location.	bin_id, device_id, latitude, longitude, status
TELEMETRY	Time-series sensor measurements.	telemetry_id, bin_id, fill_level, temperature, lat, lon, timestamp
VEHICLE	Collection vehicle resource.	vehicle_id, registration_number, capacity, status
DRIVER	Driver identity and license context.	driver_id, user_id, license_number, status
ROUTE	Collection route lifecycle and assignment.	route_id, created_by, vehicle_id, driver_id, status, version
ROUTE_STOP	Ordered bin stops in a route.	stop_id, route_id, bin_id, sequence_number, planned_time
SECURITY_EVENT	Security and audit history.	event_id, user_id/bin_id, event_type, severity, timestamp

4.3 Data integrity rules

- Primary keys and foreign keys enforce referential integrity.
- Device IDs and other identity fields use appropriate uniqueness constraints.
- Telemetry is validated before persistence.
- Route updates use state and version checks to reduce stale-write issues.
- Ordinary users cannot directly modify audit records or bypass route authorization.

4.4 DFD Level 0

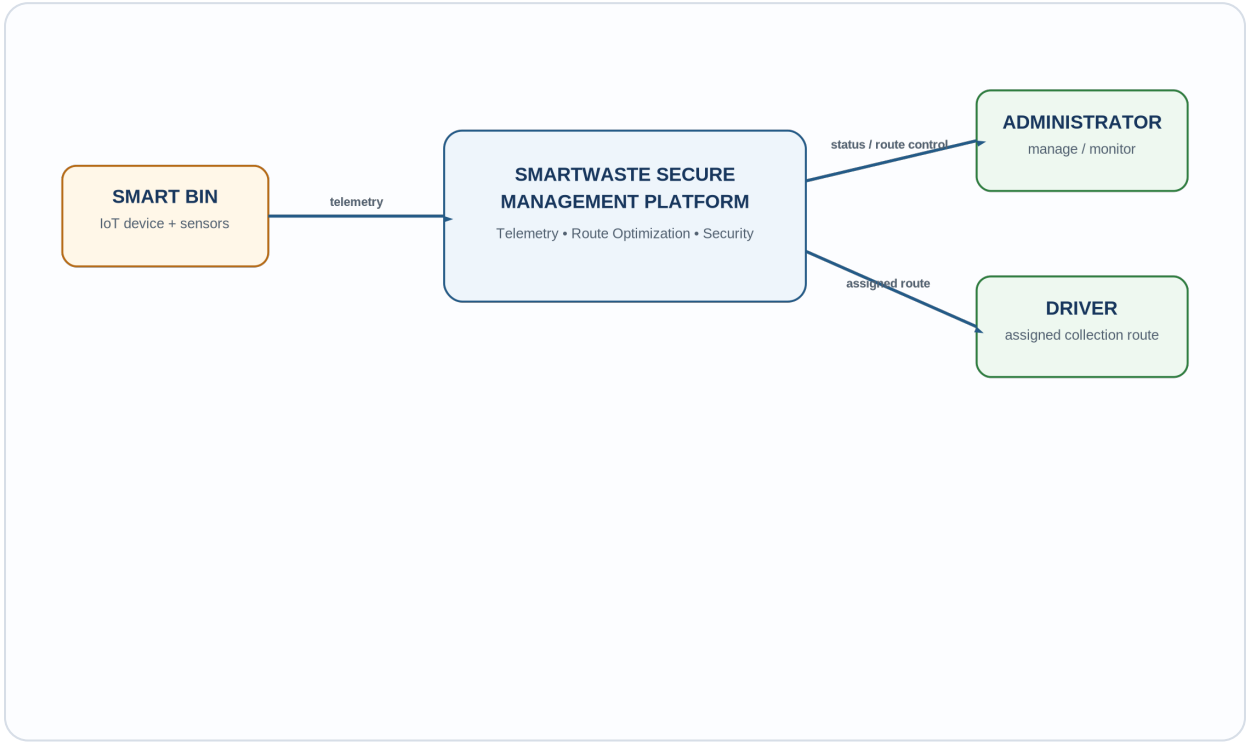


Figure 5 - DFD Level 0 - Context Diagram.

4.5 DFD Level 1

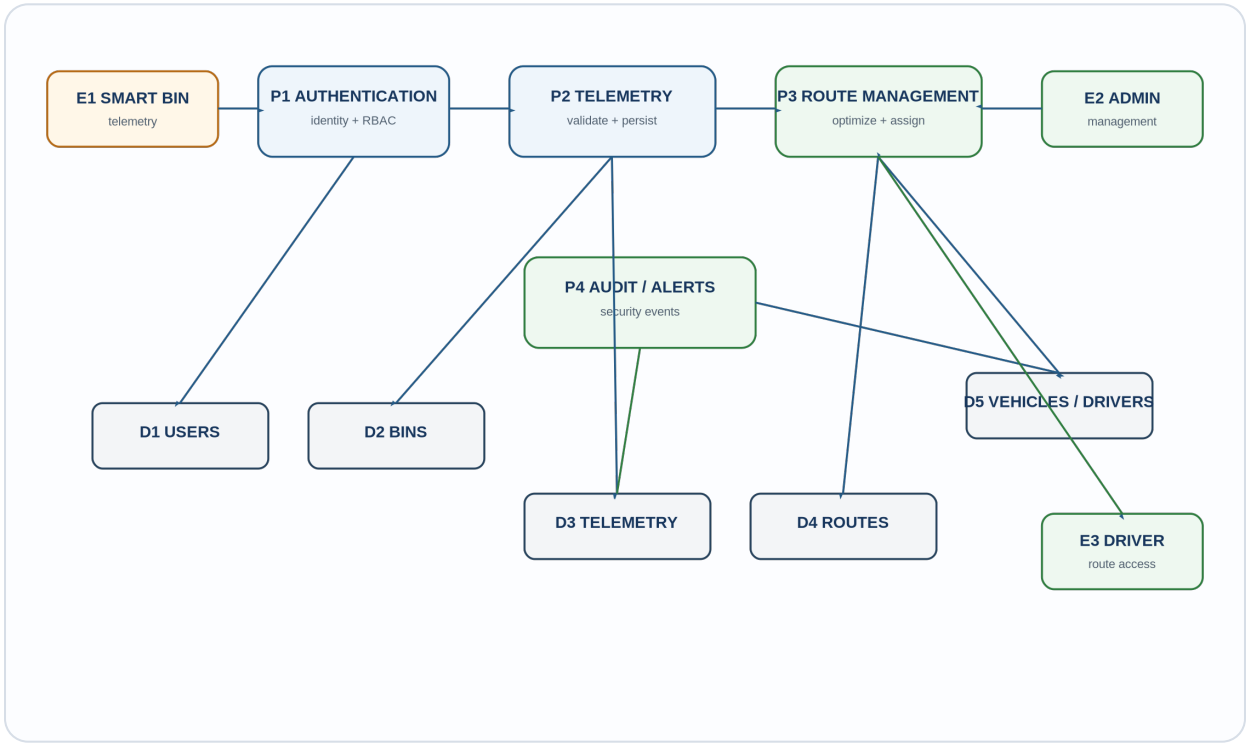


Figure 6 - DFD Level 1 - Major processes and stores.

4.6 DFD Level 2 - Telemetry

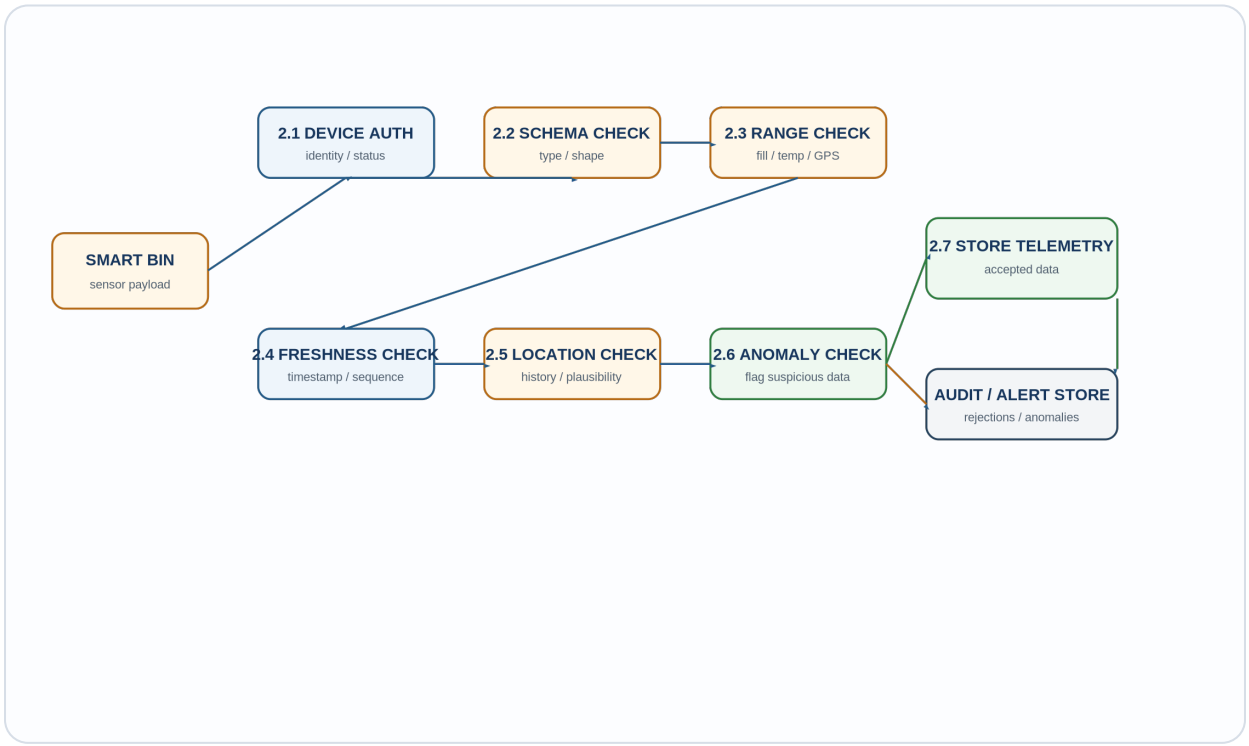


Figure 7 - DFD Level 2 - Telemetry Processing.

4.7 DFD Level 2 - Route Management

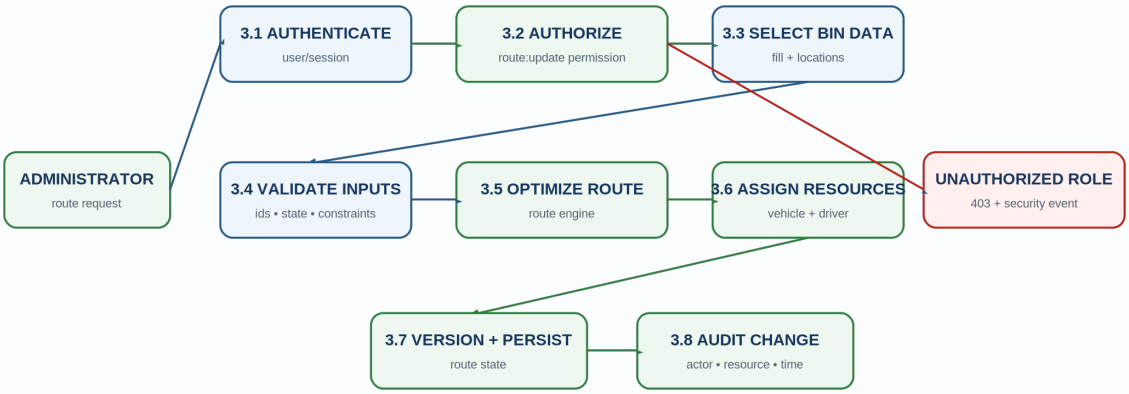


Figure 8 - DFD Level 2 - Route Management.

> Data-flow consistency rule: Smart Bin, User, Telemetry, Route, Vehicle, Driver and Security Event terminology is reused in the ERD, DFDs, architecture, requirements and later implementation.

PHASE 5 • 7 MARKS

Phase 5 - Software Architecture and Design Engineering

5.1 Architectural style

A layered modular architecture separates presentation, API entry, policy enforcement, domain services, data access and persistence. This reduces coupling and makes security controls reusable. Route optimization remains a distinct capability so untrusted input never writes directly to route state.

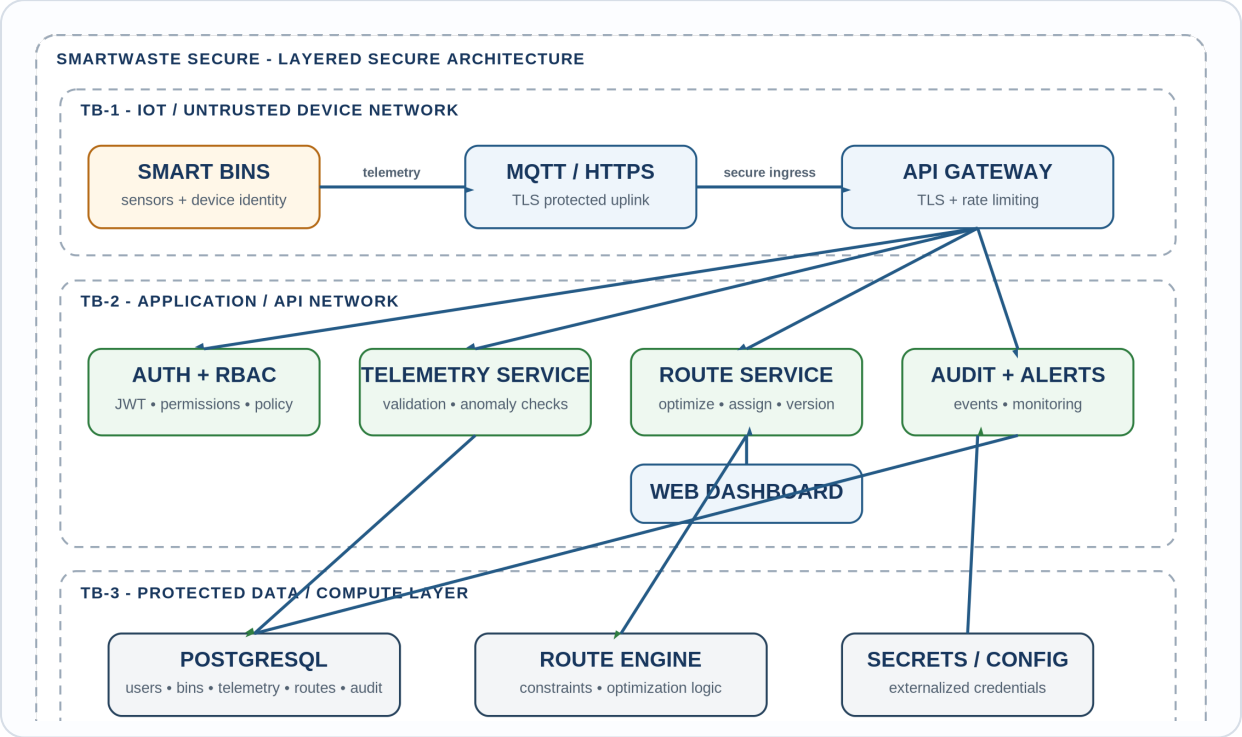


Figure 10 - Layered secure architecture with IoT, application and protected-data trust zones.

5.2 Major components and interfaces

COMPONENT	RESPONSIBILITY	PRIMARY INTERFACE / CONTROL
Web UI	Dashboard, bin monitoring, route administration and reports.	HTTPS; role-aware navigation; safe error display.
API Gateway	Controlled public entry point.	TLS; request limits; rate limiting.
Authentication + RBAC	User and device identity enforcement.	JWT/session validation; deny-by-default permissions.
Device Service	Register, activate, revoke and inspect bins.	Device identity lifecycle.
Telemetry Service	Validate, process and store measurements.	Schema, range, timestamp and anomaly checks.
Route Service	Generate, validate, version and assign routes.	Authorization; route state machine; object-level access.
Vehicle/Driver Service	Manage collection resources.	RBAC; input validation.
Audit + Alert Service	Generate structured security events and alerts.	Restricted write path; monitoring.
PostgreSQL	Persistent store.	Least-privilege DB role; network restriction.
Route Engine	Optimization under operational constraints.	Validated inputs; controlled service interface.

5.3 Design concepts / patterns

PATTERN / CONCEPT	APPLICATION
Repository Pattern	Encapsulate PostgreSQL persistence and reduce direct DB access from controllers.
Service Layer	Keep security-sensitive business rules in reusable services.
Strategy Pattern	Allow alternative route-optimization strategies without changing the API contract.

PATTERN / CONCEPT	APPLICATION
Facade Pattern	Expose a controlled route-management interface to controllers.
State-machine thinking	Prevent illegal route and device lifecycle transitions.

5.4 Hardware and IoT device baseline

The problem statement requires four periodic outputs: fill level, location, temperature and sensor status. The following is the proposed laboratory prototype that realizes those signals and demonstrates device-level security.

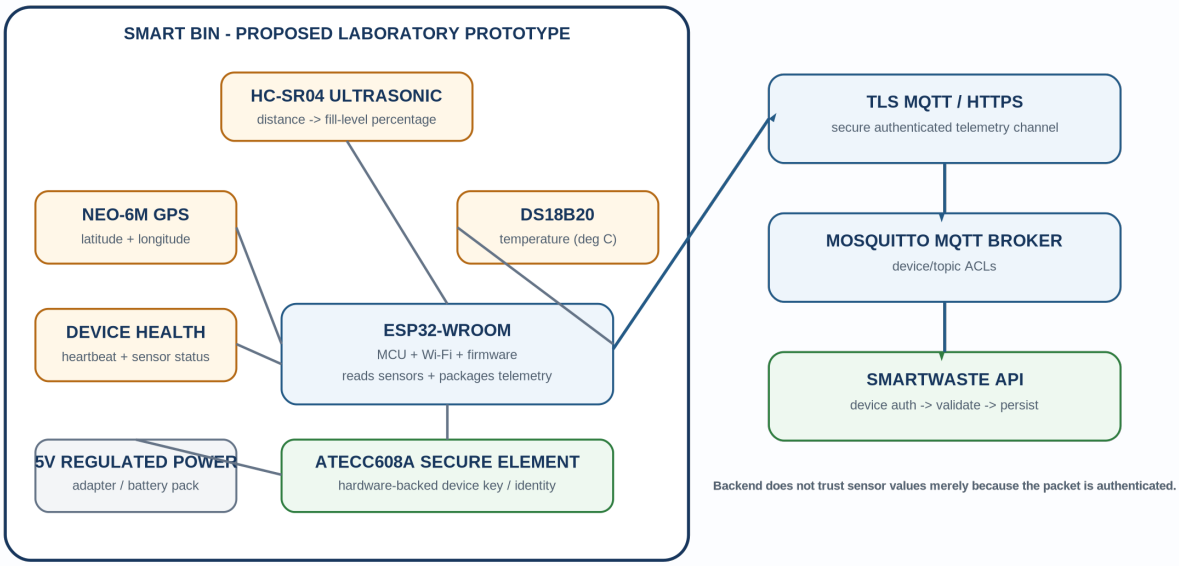


Figure 1 - Proposed smart-bin hardware and secure telemetry path.

HARDWARE / MODULE	PURPOSE	OUTPUT / FUNCTION	SECURITY RELEVANCE
ESP32-WROOM	Edge MCU with Wi-Fi.	Sensor acquisition, local packaging and secure network connection.	Unique device identity; firmware minimization; no hard-coded backend secrets.
HC-SR04 ultrasonic	Measure distance to waste surface.	Fill-level estimate.	Server-side range/plausibility checks.
NEO-6M GPS	Provide geographic coordinates.	Latitude and longitude.	Coordinate validation and movement anomaly detection.
DS18B20	Measure temperature.	Temperature in degrees C.	Range/plausibility checks; abnormal-pattern detection.
ATECC608A secure element	Protect device cryptographic identity.	Hardware-backed private-key operations.	Keys intended to remain in protected hardware.
5V regulated power	Power electronics.	Stable supply / battery-backed option.	Brownout/reset can be treated as device-health events.
Protected enclosure	Protect electronics and sensor connections.	Physical housing.	Tamper-resistant access to controller/ secure element.

HARDWARE / MODULE	PURPOSE	OUTPUT / FUNCTION	SECURITY RELEVANCE
Mosquitto MQTT broker	Laboratory message broker.	Device/topic message routing.	Broker authentication and topic ACLs.

5.5 Hardware-to-requirement mapping

PROBLEM STATEMENT OUTPUT	HARDWARE	PROCESSING	STORED FIELD
Fill level	HC-SR04	Distance -> percentage + range validation	fill_level
Location	NEO-6M GPS	Bounds + historical consistency	latitude, longitude
Temperature	DS18B20	Type/range/plausibility	temperature
Sensor status	ESP32 diagnostics	Heartbeat + sensor health	sensor_status, device_status

PHASE 6 • 5 MARKS

Phase 6 - User Interface Design

6.1 UI design

The UI follows the exam's Golden Rules: consistency, user control, feedback, error prevention, easy navigation and visibility. Role-specific screens expose only information necessary for each user class.



Figure 11 - SmartWaste Secure UI wireframes: Login, Bin Dashboard, Route Management and Security/Reports.

SCREEN	USER	GOAL	INPUTS / FEEDBACK	SECURITY CONSIDERATION
Login	Administrator / Driver / Auditor	Secure authentication.	Credentials; generic errors; lockout/throttling feedback.	TLS, secure tokens, no secret disclosure.
Bin Dashboard	Administrator / Operator	Identify bin condition and priority.	Fill, temperature, location, status, alerts.	Role-restricted data and safe rendering.
Route Management	Administrator	Generate, assign and modify routes.	Bins, vehicle, driver, route state.	RBAC/object authorization; audited updates.
Driver Route	Driver	Follow assigned collection sequence.	Stops, status updates, issue report.	Access only to assigned route.
Security / Reports	Auditor / Administrator	Review anomalies and audit events.	Filters, severity, event details.	Read-only auditor role; sensitive fields redacted.

6.2 Error prevention

- Disable route mutation controls for unauthorized roles.
- Validate route state before enabling save/assign actions.
- Show clear but non-sensitive API errors.
- Confirm high-impact route changes and display audit reference.

PHASE 7 • 10 MARKS

Phase 7 - Threat Modeling and Security Analysis

7.1 Asset and CIA analysis

A-01

User credentials

C: High | I: High | A: Medium

Compromise enables account takeover.

A-02

Device credentials

C: High | I: Critical | A: High

Fake devices could inject telemetry.

A-03

Bin telemetry

C: Medium | I: High | A: High

Manipulated values can affect collection decisions.

A-04

Bin location

C: High | I: Critical | A: High

Incorrect coordinates can corrupt routes.

A-05

Collection routes

C: High | I: Critical | A: High

Unauthorized changes can disrupt operations.

A-06

Vehicle information

C: Medium | I: High | A: High

Required for route assignment.

A-07

Driver information

C: High | I: High | A: Medium

Contains identity and operational data.

A-08

Audit/security logs

C: High | I: Critical | A: High

Security evidence must remain trustworthy.

A-09

Route optimization data

C: Medium | I: Critical | A: High

Incorrect data produces poor routes.

A-10

Security alerts

C: High | I: High | A: High

Delayed alerts reduce incident response.

A-11

Application secrets

C: Critical | I: Critical | A: High

Exposure can compromise services and data.

A-12

PostgreSQL database

C: High | I: Critical | A: Critical

Central operational data store.

7.2 STRIDE threat analysis

DFD ELEMENT	THREAT	STRIDE	IMPACT	MITIGATION
Smart Bin -> API/ MQTT	Fake device identity	Spoofing	Critical	Unique device identity, credential lifecycle, TLS.
Telemetry Service	Modified fill level	Tampering	High	Schema/range validation; anomaly detection.
Telemetry / location	Manipulated coordinates	Tampering	Critical	Coordinate checks; historical consistency.
API Gateway	Data exposure in transit	Information Disclosure	High	TLS; safe error responses.
Route Service	Unauthorized route modification	Elevation / Tampering	Critical	JWT/RBAC; object-level authorization; audit.
Telemetry endpoint	Flooded requests	Denial of Service	High	Rate limiting; quotas; resource limits.
MQTT broker	Unauthorized publish	Spoofing / Tampering	Critical	Broker authentication and topic ACLs.
Audit Service	Deletion/modification of records	Repudiation / Tampering	High	Restricted writes; append-oriented records.
Secrets / config	Credential leakage	Information Disclosure	Critical	Secret management; repository scanning.
Kubernetes workload	Excessive privileges	Elevation of Privilege	High	Non-root; allowPrivilegeEscalation=false.
Device lifecycle	Revoked device continues sending	Spoofing	High	Server-side status/revocation check.
Route API	Object ID guessing	Information Disclosure	Medium	Authorization on every object; opaque IDs where useful.

7.3 Information flow analysis

SENSITIVE ASSET	SOURCE	PROCESS	STORE / DESTINATION	UNAUTHORIZED FLOW	CONTROL
Bin location	Smart Bin	Telemetry Processing	Telemetry Store -> Route Engine	Forged GPS or replayed location	TLS; device auth; coordinate validation; anomaly checks.
Collection route	Administrator	Route Management	Route Store -> Driver	Unauthorized route modification	JWT/RBAC; object auth; versioning; audit.
Device credentials	Provisioning / device	Authentication	Identity store / secure element	Credential theft or cloning	Hardware-backed key storage; rotation/revocation; no hard-coded secrets.
Audit events	Application services	Audit / Alerts	Audit Store -> Auditor	Log leakage or alteration	Restricted access; append-oriented records; redaction.

7.4 Vulnerability analysis

ID	VULNERABILITY	AFFECTED ELEMENT	RELATED THREAT	IMPACT	MITIGATION
V-01	Missing / weak device authentication	MQTT/API	Sensor spoofing	Critical	Unique device identity; revocation; authentication.
V-02	Telemetry values accepted without range validation	Telemetry API	Sensor spoofing / tampering	High	Schema and range validation.
V-03	Location updates accepted without plausibility checks	Telemetry API	Location manipulation	Critical	Coordinate and movement anomaly checks.
V-04	Broken object-level authorization on route IDs	Route API	Unauthorized route modification	Critical	Server-side object authorization.
V-05	No rate limit on telemetry/API	Gateway	Availability / abuse	High	Per-device/IP throttling and quotas.
V-06	Hard-coded or exposed credentials	Source/config	Secret exposure	Critical	Environment/Kubernetes Secret; Gitleaks.
V-07	Database account has excess privileges	Database	Privilege escalation	High	Dedicated least-privilege DB role.
V-08	Sensitive errors reveal internals	API	Information disclosure	Medium	Safe messages; server-side detailed logs.
V-09	Container runs as root	Container	Privilege escalation	High	Non-root user/securityContext.
V-10	Unrestricted service exposure	Kubernetes	Exposure/availability	High	ClusterIP or controlled ingress.

> **Key security premise:** an authenticated smart bin is not automatically trusted. Authentication establishes device identity; server-side validation and anomaly checks determine whether its reported data is safe to use.

PHASE 8 • 6 MARKS

Phase 8 - Attack Tree and Security Architecture Refinement

8.1 Critical attacker goal

Root goal: manipulate waste-collection operations by spoofing devices, corrupting location data, modifying routes or disrupting platform availability.

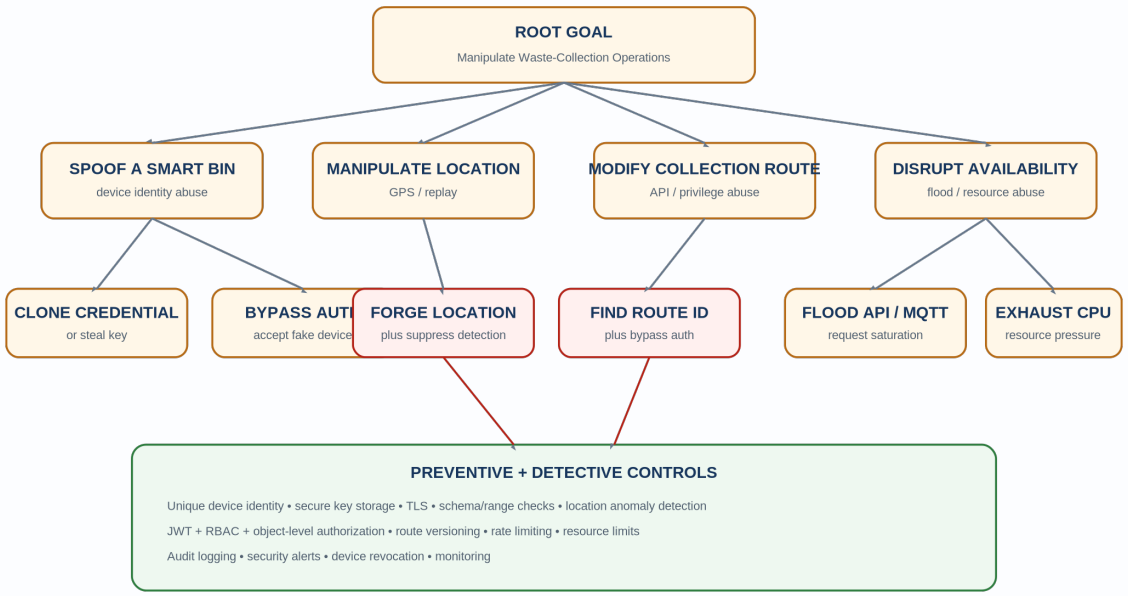


Figure 12 - Attack Tree. OR branches represent alternative paths; highlighted AND branches require multiple conditions.

8.2 Security architecture refinement driven by the attack tree

HIGHEST-RISK PATH	ARCHITECTURE REFINEMENT	CONTROL TYPE
Spoof a smart bin	Device registry + secure identity + revocation before telemetry acceptance.	Preventive + detective
Manipulate location	Location validation + history-based anomaly detection before route optimization.	Preventive + detective
Modify route	Dedicated route permission + object authorization + route versioning + audit.	Preventive + detective
Disrupt availability	Rate limits + broker ACLs + Kubernetes resource limits + health checks.	Preventive + resilience

PHASE 9 • 7 MARKS

Phase 9 - Product Backlog and Jira / Scrum

9.1 Product backlog

ID	EPIC	USER STORY	PRIORITY	POINTS
US-01	EP-01 Identity	As an administrator, I want to authenticate securely so that only authorized users can manage the platform.	Critical	5
US-02	EP-02 Device	As an administrator, I want to register and revoke smart bins so that device access can be controlled.	High	5
US-03	EP-03 Telemetry	As a system, I want to authenticate each bin so that unknown devices cannot submit telemetry.	Critical	5
US-04	EP-03 Telemetry	As a system, I want to validate telemetry so that invalid readings do not affect operations.	Critical	5
US-05	EP-04 Monitoring	As an administrator, I want to view bin status so that I can prioritize collection.	High	3
US-06	EP-05 Routes	As an administrator, I want to generate optimized routes so that collection is efficient.	Critical	5
US-07	EP-05 Routes	As an administrator, I want to assign vehicles and drivers so that routes can be executed.	High	5
US-08	EP-05 Routes	As a driver, I want to view my assigned route so that I know my collection sequence.	High	3
US-09	EP-06 Security	As a system, I want to detect suspicious telemetry so that spoofing can be investigated.	Critical	5
US-10	EP-06 Security	As an administrator, I want route changes to require authorization so that unauthorized modifications are blocked.	Critical	5
US-11	EP-07 Audit	As a security auditor, I want security events recorded so that suspicious activity can be investigated.	High	3
US-12	EP-08 Availability	As an operator, I want rate limiting and health monitoring so that service abuse is contained.	High	3
US-13	EP-09 Resources	As an administrator, I want to manage vehicles and drivers so that assignments remain accurate.	High	3
US-14	EP-10 Testing	As a security engineer, I want automated security tests so that regressions are detected before release.	High	5

9.2 Common acceptance criteria for critical stories

- ✓ Unauthorized roles receive 403 and no persistent state change.
- ✓ Invalid telemetry is rejected before route-related processing.
- ✓ Accepted telemetry and sensitive administrative changes are auditable.
- ✓ Security failures do not expose credentials, tokens or stack traces.

9.3 Jira/Scrum structure

JIRA ELEMENT	SMARTWASTE DEFINITION
Project	SmartWaste Secure
Project key	SWS

JIRA ELEMENT	SMARTWASTE DEFINITION
Project type	Scrum
Epics	Identity, Device, Telemetry, Monitoring, Routes, Security, Audit, Resources, Testing
Issue types	Epic, Story, Task, Bug
Required board	TO DO -> IN PROGRESS -> TESTING -> DONE

PHASE 10 • 7 MARKS

Phase 10 - Sprint Execution and Scrum Metrics

10.1 Two-sprint academic plan

SPRINT	GOAL	PRIMARY SCOPE
Sprint 1 - Secure IoT Foundation	Build identity, device registration, telemetry validation, storage and dashboard foundation.	US-01 to US-05, device/auth services, telemetry schema, database, dashboard, unit tests.
Sprint 2 - Secure Route Operations	Deliver route optimization, assignment, auditability, deployment and security assurance.	US-06 to US-14, route service, security controls, Docker, Kubernetes, CI/CD, fuzzing and regression tests.

10.2 Required board flow

TO DO -> IN PROGRESS -> TESTING -> DONE

10.3 Daily Scrum template

ITEM	ENTRY FORMAT
Yesterday	Completed work items / tests.
Today	Planned tasks with story IDs.
Blockers	Environment, access, dependency or security blockers.

10.4 Metrics integrity

> **No live Jira metrics are asserted here.** Velocity, burndown, defects and carry-over must be populated from the actual Jira workspace after task execution. Planned sprint points are specification artifacts, not observed performance.

METRIC	DEFINITION	EVIDENCE
Velocity	Completed story points per sprint.	Jira sprint report
Burndown	Remaining story points by day.	Jira burndown
Defects	Functional and security bugs raised/resolved.	Jira bug issues
Carry-over	Incomplete work moved to next sprint.	Sprint completion report

METRIC	DEFINITION	EVIDENCE
Review outcome	Accepted, rejected and changed work.	Sprint review notes
Retrospective actions	Improvements selected for next sprint.	Retro notes

PHASE 11 • 6 MARKS

Phase 11 - Secure Development and Build Environment

11.1 Repository and branch workflow

```
main
  |
  +-- develop
      +-- feature/device-auth
      +-- feature/telemetry
      +-- feature/routes
      +-- feature/audit
      +-- security/fixes
```

11.2 Secure-build controls

CONTROL	IMPLEMENTATION
Least privilege	Developer, CI and application identities receive only required access.
Secret management	Environment variables, Kubernetes Secrets and CI secret stores; never source code.
Dependency control	Pinned or constrained dependencies plus pip-audit/SCA scanning.
Code review	Pull-request review for security-sensitive changes.
Protected branches	Require review and passing checks before merging to main.
Reproducible builds	Pinned dependencies and Docker build definition.
Artifact integrity	Image tags/digests and controlled release path.

11.3 Automated static/security checks

CHECK	PURPOSE	EXAMPLE COMMAND
Bandit	Python static security patterns.	bandit -r app/
pip-audit	Known Python dependency vulnerabilities.	pip-audit
Gitleaks	Detect hard-coded secrets.	gitleaks detect
Trivy	Container/package vulnerabilities.	trivy image smartwaste:latest

> **Evidence requirement:** capture a finding, remediate it, rerun the tool and retain both pre-fix and post-fix results.

PHASE 12 • 4 MARKS

Phase 12 - Secure Coding and Refactoring

12.1 Secure coding module - route modification API

The route-update endpoint is selected because it is the clearest demonstration of authentication, authorization, input validation, error handling and auditing.

```
@app.route("/api/routes/<int:route_id>", methods=["PUT"])
def update_route(route_id):
    route = Route.query.get(route_id)
    route.status = request.json["status"]
    db.session.commit()
    return jsonify({"message": "updated"})
```

Initial weaknesses: no authentication, no authorization, unsafe input access, no audit event, weak error handling and IDOR exposure.

```
@app.route("/api/routes/<int:route_id>", methods=["PUT"])
@jwt_required()
@require_permission("route:update")
def update_route(route_id):
    data = request.get_json(silent=True)
    if not data:
        return jsonify({"error": "Invalid request"}), 400

    route = Route.query.get(route_id)
    if route is None:
        return jsonify({"error": "Route not found"}), 404

    status = data.get("status")
    allowed = {"PLANNED", "ASSIGNED", "IN_PROGRESS", "COMPLETED"}
    if status not in allowed:
        return jsonify({"error": "Invalid status"}), 400

    route.status = status
    route.version += 1
    db.session.commit()
    create_audit_event("ROUTE_MODIFIED", route_id)

    return jsonify({"message": "Route updated"})
```

12.2 Security improvement

WEAKNESS	FIX	BENEFIT
No authentication	JWT/session authentication.	Reject anonymous modification attempts.
No authorization	Permission + object-level checks.	Driver cannot modify another route.
Unchecked input	Schema/state validation.	Reject malformed and illegal states.
No audit	Structured route-modification event.	Accountability and investigation.
Weak error handling	Safe status-specific responses.	Reduces information leakage.

PHASE 13 • 7 MARKS

Phase 13 - Containerized Development: Docker and Kubernetes

13.1 Container strategy

```
FROM python:3.12-slim
RUN useradd --create-home appuser
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
RUN chown -R appuser:appuser /app
USER appuser
EXPOSE 5000
```

DOCKER SECURITY PRACTICE	IMPLEMENTATION
Minimal image	python:3.12-slim rather than a full OS image.
Non-root	Run application as appuser.
Controlled ports	Expose only the API port required by the lab.
Version control	Pin dependencies and use deterministic build files.
Secret handling	Inject at runtime rather than baking secrets into image.
Package minimization	Do not install unnecessary OS packages.

13.2 Kubernetes security controls

```
securityContext:
  runAsNonRoot: true
  allowPrivilegeEscalation: false

resources:
  requests:
    cpu: "100m"
    memory: "128Mi"
  limits:
    cpu: "500m"
    memory: "512Mi"
```

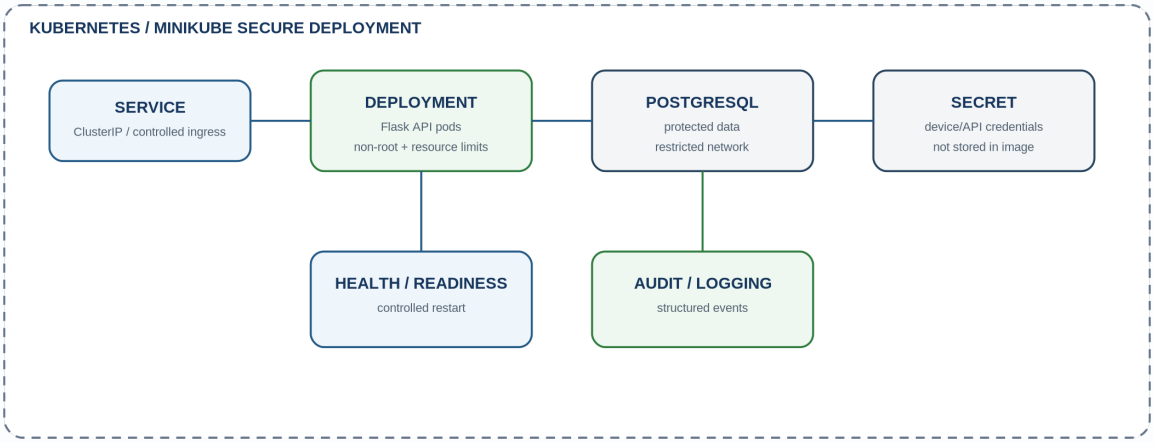


Figure 13 - Kubernetes/Minikube deployment with restricted service exposure, non-root workload, resources and protected configuration.

KUBERNETES CONTROL	PURPOSE
Namespace isolation	Separate application objects from unrelated workloads.
Non-root securityContext	Reduce impact of application compromise.
Resource limits	Contain CPU/memory exhaustion.
Restricted Service exposure	Avoid unnecessary direct public access.
Secret management	Keep credentials outside container images.

PHASE 14 • 7 MARKS

Phase 14 - CI/CD and Security Testing

14.1 CI/CD pipeline

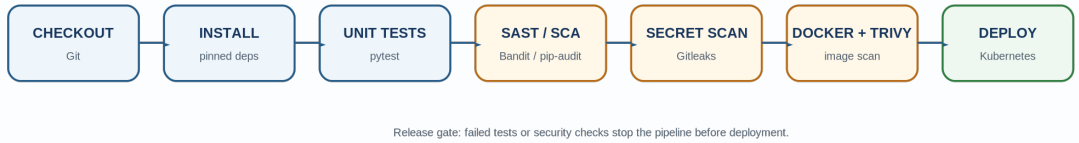


Figure 14 - Secure CI/CD pipeline and release gate.

14.2 Test strategy

TEST LEVEL	SCENARIO / FOCUS	EXPECTED EVIDENCE
Unit	Telemetry validator; permission policy; route-state validation.	pytest pass/fail output.
Integration	Device/auth -> telemetry -> DB; admin/auth -> route -> audit.	API/integration test output.
System / E2E	Login -> dashboard -> route generation -> assignment -> driver view.	End-to-end run evidence.
Security regression	Unauthorized route change; invalid device; bad coordinates; rate limit; secret scan.	Repeatable security test results.
Fuzzing	Malformed telemetry JSON, extreme numeric inputs, unexpected fields.	Fuzz execution log and observations.

14.3 Fuzzing boundary

INPUT	EXAMPLE MALICIOUS / INVALID VALUE	EXPECTED
fill_level	-999 / 999999 / string	Reject with validation error.
latitude	999 / null / malformed string	Reject.
longitude	-999 / null	Reject.
temperature	NaN / extreme value	Reject or flag depending on policy.
timestamp	stale / future / invalid	Reject or flag according to freshness policy.
payload	empty / very large JSON / unknown fields	Safe rejection within request limits.

14.4 Defect and retest template

FIELD	EXAMPLE RECORD
Defect	DEF-001 - telemetry accepts fill level > 100.
Severity	High.
Expected	400 Bad Request.
Initial actual	Value accepted - defect open.
Fix	Add server-side range validation.
Retest	Same input rejected with 400 - defect closed.

PHASE 15 • 5 MARKS

Phase 15 - Logging, Monitoring, Hardening and Secure Deployment

15.1 Security-relevant events

EVENT	WHY LOG IT	DO NOT LOG
LOGIN_FAILURE	Detect credential abuse.	Password / secret.
DEVICE_AUTH_FAILURE	Detect spoofing or revoked device use.	Private key.
TELEMETRY_REJECTED	Investigate invalid/suspicious input.	Unnecessary sensitive payload data.
LOCATION_ANOMALY	Investigate possible spoofing.	Secrets/tokens.
ROUTE_MODIFIED	Provide accountability for high-impact changes.	Session tokens.
PRIVILEGE_CHANGE	Trace role/permission changes.	Credentials.
RATE_LIMIT_TRIGGERED	Detect API abuse and DoS indicators.	Authorization headers.

15.2 Monitoring metrics and alerts

METRIC	ALERT CONDITION	ACTION
API error rate	Above baseline.	Investigate service health.
Authentication failures	Sudden spike.	Investigate credential/device abuse.
Device auth failures	Repeated failures from same source/device.	Review and potentially revoke.
Telemetry rejection rate	Unexpected increase.	Investigate sensor/network issues.
Offline bins	Count above threshold.	Operational follow-up.
Unauthorized route attempts	Any unexpected sensitive denial.	Security investigation.
CPU / memory pressure	Near configured limit.	Scale / remediate / investigate.

15.3 Hardening checklist

Access
Least privilege; strong credentials; protected admin routes; periodic access review.

Network
TLS; minimal exposed ports; restricted DB access; controlled MQTT broker exposure.

Secrets
No secrets in source or images; rotate/revoke device credentials; protect CI secrets.

Runtime
Non-root containers; resource limits; health checks; patched base image.

Database
Dedicated DB user; minimum privileges; backup and restore procedure.

Operations
Incident response; monitoring; log review; device revocation and replacement procedure.

15.4 Physical and operational IoT controls

- Tamper-resistant enclosure around the ESP32 and secure element.
- Restricted physical access to device provisioning interfaces.
- Secure device replacement and revocation procedure.
- Controlled firmware update process; future extension: signed firmware validation.
- Power and reset events captured as device-health observations.

PHASE 16 • 1 MARK

Phase 16 - Final Security Review

16.1 End-to-end traceability

REQUIREMENT	USE CASE	DFD / DATA	THREAT	VULNERABILITY	USER STORY	VERIFICATION	DEPLOYMENT CONTROL
SEC-04 Telemetry validation	UC-01 Submit Telemetry	P2 Telemetry -> D3 Telemetry	Sensor spoofing / tampering	V-02 Weak validation	US-04	Unit + fuzz test	API validation middleware
SEC-05 Location integrity	UC-01 Submit Telemetry	P2 Telemetry -> P3 Route	Location manipulation	V-03 Weak location checks	US-09	Location anomaly test	Validated route inputs
SEC-03 Route authorization	UC-02 Modify Route	P3 Route Management -> D4 Routes	Unauthorized route modification	V-04 BOLA/IDOR	US-10	Unauthorized role receives 403	JWT/RBAC + audit
SEC-10 Audit	UC-02 / security events	P4 Audit / Alerts	Repudiation	Audit tampering risk	US-11	Audit event regression	Restricted audit storage
SEC-07 Availability	Telemetry/ API ingress	Gateway -> services	DoS	V-05 Missing rate limits	US-12	Abuse/rate-limit test	Rate limit + K8s resources

16.2 Three highest-risk issues

RISK	WHY HIGHEST	CONTROL
Unauthorized route modification	Direct operational impact and high integrity consequence.	RBAC, object authorization, route versioning, audit.
Sensor/location spoofing	Can corrupt route optimization and physical collection decisions.	Device authentication, TLS, validation, anomaly detection.
Availability attack	Can prevent telemetry collection and route operations.	Rate limiting, quotas, health checks, resource limits, monitoring.

16.3 Remaining limitations / future improvements

- GNSS spoofing cannot be eliminated by software-only checks; future versions can combine trusted location signals and stronger hardware attestation.
- A local Minikube environment is not equivalent to production municipal infrastructure; future deployment can use managed Kubernetes, centralized SIEM, HSM-backed secrets and high-availability database services.

CLOSING CHAPTER

Conclusion and Future Work

SmartWaste Secure unifies the operational needs of an IoT waste-collection platform with secure-software engineering practices. The design begins with the smart-bin problem statement and carries the same vocabulary through requirements, use cases, the ER model, DFDs, trust boundaries, threat analysis, backlog, implementation controls and verification.

The strongest security decision is to treat telemetry as untrusted input even after device authentication. Location and fill data are validated before they can influence route optimization, while route state changes require explicit authorization and auditability. The container and Kubernetes design extend these controls into the runtime environment.

Future work

- Hardware-backed attestation and signed firmware updates for smart bins.
- Multi-source location integrity checking and stronger anti-spoofing techniques.
- Centralized SIEM integration and long-term security analytics.
- High-availability production deployment with managed Kubernetes and database services.
- More extensive load, concurrency and fault-injection testing.

APPENDICES

Appendix A - Security Requirements Register

ID	SECURITY REQUIREMENT	VERIFICATION
SEC-01	Device Authentication	Every registered smart bin shall have a unique authenticated device identity.
SEC-02	Secure Transport	Telemetry and application traffic shall use TLS-protected channels.
SEC-03	Authorization	Route modifications and administrative functions require server-side RBAC/object authorization.
SEC-04	Input Validation	Telemetry and route inputs shall be schema, type, range and state validated.
SEC-05	Location Integrity	Coordinates shall be checked for valid ranges, impossible movement and suspicious change.
SEC-06	Replay Resistance	Telemetry shall use timestamp/sequence information to reject stale or repeated messages where implemented.
SEC-07	API Protection	Sensitive endpoints shall be rate-limited and return safe errors.
SEC-08	Secret Management	Credentials and keys shall not be hard-coded or committed to source control.
SEC-09	Least Privilege	The IoT device and application database account shall have only required permissions.
SEC-10	Audit	Login failures, privilege changes, route modifications and anomalous telemetry shall be logged.
SEC-11	Container Security	Application containers shall run without unnecessary privileges.
SEC-12	Kubernetes Security	Deployment shall apply non-root context, resource limits and restricted exposure.

Appendix B - Security Test Matrix

TEST ID	SCENARIO	EXPECTED BEHAVIOR	VERIFICATION
T-01	Invalid device credential	Telemetry rejected; event logged.	Unit / integration
T-02	Fill level outside 0-100	400; no persistence.	Unit / fuzz
T-03	Impossible GPS coordinates	400 or flagged according to policy.	Unit / security regression
T-04	Stale / repeated telemetry	Rejected where freshness/sequence checks apply.	Integration
T-05	Driver attempts route modification	403; no state change; security event.	Negative authorization test
T-06	API request flood	Rate limited; service remains responsive.	Abuse test
T-07	Secret committed to source	CI secret scan fails.	Gitleaks
T-08	Root container	Build/policy check fails if root.	Container security check
T-09	Unauthorized Kubernetes exposure	Service not directly exposed beyond intended boundary.	Deployment review

Appendix C - Reference / Evidence Policy

This report follows the structure and evidence-oriented presentation of the supplied Secure Digital Voting System capstone report: cover and document control, abstract, table of contents, lists of figures/tables, numbered engineering chapters, diagrams with captions, requirement/security registers, Scrum planning, DevSecOps,

traceability, conclusion and appendices. The reference report explicitly treats the engineering lifecycle as a chain from requirements through use cases, data models, DFDs, threat analysis, attack tree, backlog, Jira and verification.

> **Integrity note:** planned architecture, tests, Jira scope and metrics are described as design specifications until corresponding live evidence is captured. This follows the evidence-integrity approach in the supplied reference report, which does not assert live Jira metrics before actual workspace capture.

References

1. 24CYS401 Secure Software Engineering End Semester Laboratory Examination - supplied course instructions.
2. Secure Digital Voting System Capstone Report, supplied academic reference report, October 2026.
3. OWASP Foundation - OWASP Top 10 and OWASP API Security Top 10.
4. NIST SP 800-218 - Secure Software Development Framework (SSDF).
5. NIST SP 800-207 - Zero Trust Architecture.

EVIDENCE CHECKLIST

Final Submission Checklist

REQUIRED ARTIFACT	INCLUDED IN REPORT / PLANNED EVIDENCE
SRS and security requirements	Phases 2 and Appendix A
Use Case Diagram + specifications + analysis model	Phase 3
ER Diagram + DFD + trust boundaries	Phase 4
Architecture/component design + UI	Phases 5 and 6
Asset/CIA + STRIDE + information flow + vulnerability analysis	Phase 7
Attack Tree	Phase 8
Product Backlog + Jira + two Sprint Plans	Phases 9 and 10
Sprint Board + Daily Scrum + Burndown + Velocity + Defects + Review/ Retrospective	Phase 10; live evidence to be attached after execution
Source code + refactoring/security fixes	Phase 12
Dockerfile + container + Kubernetes deployment evidence	Phase 13
CI/CD + unit/integration/system tests + fuzzing	Phase 14
Logging/monitoring + hardening/deployment checklist	Phase 15
Final traceability and security review	Phase 16